

Namespace PhotoCatalog.Application.

Caching

Classes

[CacheKeysFactory](#)

Централизованная фабрика строковых идентификаторов для кэширования.
Разделяет генерацию Key и Tag.

Полное Техническое Задание на разработку библиотеки каталогизации фотографий

1. Назначение документа и описание проекта

Назначение: Документ описывает архитектуру, функциональные требования и структуру компонентов для системы каталогизации фотографий.

Суть проекта: Разработка ядра (библиотеки) для управления коллекцией фотографий с использованием концепции "виртуальных альбомов". Физические файлы хранятся в единой директории, а иерархия папок, альбомов и тегов формируется исключительно на логическом уровне в базе данных.

2. Архитектурные требования и стек технологий

Система проектируется с соблюдением принципов **Чистой Архитектуры**, **SOLID** и **Богатой Доменной Модели**.

- **Основная платформа:** .NET 10
- **Язык программирования:** C# 13
- **База данных:** SQLite
- **Доступ к данным: Dapper.** Используется как легковесный ORM для выполнения SQL-запросов и маппинга результатов в доменные объекты, обеспечивая баланс между производительностью, контролем над SQL и удобством разработки.
- **Логирование: Serilog.** Используется для структурированного логирования бизнес-потоков, отслеживания действий пользователя и фиксации системных сбоев.
- **Тестирование:** xUnit (Unit и интеграционные тесты), ArchUnitNET (архитектурные тесты).
- **Инверсия зависимостей (DIP):** Внутренние слои (**Domain**) абсолютно независимы от внешних (**Infrastructure**, **Application**, сторонние библиотеки). Взаимодействие происходит строго через контракты (интерфейсы).

3. Функциональные требования

3.1. Управление физическими файлами

- **Импорт:** Система принимает путь к файлу, вычисляет его хэш, извлекает метаданные (размер, ширина, высота) и копирует/перемещает его в единую целевую директорию хранилища.

- **Синхронизация:** Система проверяет наличие физического файла по сохраненному пути и помечает записи в БД как "осиротевшие", если файл был удален средствами ОС.
- **Физическое удаление:** При явном удалении фотографии из системы, виртуальные связи в БД уничтожаются каскадно в рамках транзакции. Физический файл удаляется с диска строго **после** успешного коммита транзакции в базе данных.

3.2. Управление виртуальной иерархией

- **Виртуальные папки:** Создание, переименование, удаление и поддержка вложенности (папка внутри папки). На диске эти папки не создаются. Защита от создания циклических зависимостей (помещение папки внутрь своего потомка) контролируется на слое *Application*.
- **Виртуальные альбомы:** Создание альбомов внутри папок или в корне виртуального дерева.
- **Управление контентом:** Добавление фотографии в альбом (создание логической связи). Одна фотография может одновременно находиться в неограниченном количестве альбомов. Удаление фото из альбома удаляет только связь, физический файл остается нетронутым.

3.3. Тегирование и поиск

- **Тегирование:** Создание строковых тегов и привязка их к фотографиям.
- **Поиск и фильтрация:** Получение списка фотографий по пересечению условий:
 - Нахождение в определенном виртуальном альбоме или папке.
 - Наличие всех или хотя бы одного из указанных тегов.

4. Структура проекта (Слои)

Директории (папки) в решении должны именоваться строго в **PascalCase**. Проект разделен на три основных слоя:

4.1. PhotoCatalog.Domain (Ядро)

Не имеет никаких внешних зависимостей. Содержит чистую бизнес-логику.

Свободен от любых библиотек логирования.

- **Entities:** *Photo*, *Album*, *Folder*, *Tag*. Инкапсулируют состояние (используются *private* или *init-only* сеттеры). Логика изменения состояния находится внутри самих классов и возвращает объекты *Result* / *ResultVoid* вместо выброса исключений при нарушении бизнес-правил.

- **Value Objects:** Неизменяемые объекты без идентичности (например, `Dimensions` для габаритов фото).
- **Errors Registry:** Реестр доменных ошибок для контроля бизнес-правил и инвариантов (например, `DomainErrors.Tag.TooLong`).
- **Interfaces:** `IPhotoRepository`, `IFolderRepository`, `IFileStorage`, `IUnitOfWork`. Контракты репозитория возвращают `Result<T>` или `ResultVoid` для обеспечения консистентности ROP.

4.2. PhotoCatalog.Application (Сценарии использования)

Зависит только от `Domain`. Является основным местом для логирования бизнес-потоков.

- **Use Cases (Services):** Обработчики команд (например, `ImportPhotoUseCase`). Оркестрируют логику, **логируют** начало важных операций, их успешное завершение и фиксируют провальные `Result` (с уровнями Warning/Information) без остановки приложения.
- **DTO:** Простые структуры данных для возврата результатов на уровень представления.

4.3. PhotoCatalog.Infrastructure (Реализация)

Зависит от `Domain` и `Application`. Отвечает за логирование критических системных сбоев.

- **Data Access:** Реализация репозитория для SQLite с использованием **Dapper**. Реализация `IUnitOfWork` инкапсулирует соединение с базой данных для контроля его жизненного цикла.
- **File System:** Реализация `IFileStorage` через классы `System.IO`.
- **Перехват исключений:** Любые `SqlException`, `IOException` и другие системные ошибки отлавливаются здесь, **логируются через Serilog** (с уровнем Error/Fatal и полным StackTrace), а затем безопасно конвертируются в `Result.Failure(InfrastructureErrors...)`.

5. Проектирование базы данных SQLite

Схема спроектирована для эффективной работы с реляционными связями:

- `Photos`: Id, RealPath (UNIQUE), FileHash, Width, Height, AddedAt.
- `Folders`: Id, Name, ParentFolderId (FK -> Folders.Id).
- `Albums`: Id, Name, FolderId (FK -> Folders.Id).
- `Tags`: Id, Name (UNIQUE).

- **AlbumPhotos**: AlbumId, PhotoId (Composite PK, ON DELETE CASCADE).
- **PhotoTags**: PhotoId, TagId (Composite PK, ON DELETE CASCADE).

Обязательное требование: При каждом открытии соединения в **Infrastructure** необходимо выполнять **PRAGMA foreign_keys = ON**; для активации ссылочной целостности SQLite. Этот вызов контролируется внутри реализации **IUnitOfWork**.

6. Нефункциональные требования к коду и тестированию

- **Стиль кода:** Строгое соблюдение официального Style Guide от Microsoft (C#).
- **Архитектурные тесты (ArchUnitNET):** Наличие тестов, проверяющих направления зависимостей (**Domain** не знает про **Infrastructure** и **Serilog**).
- **Unit-тесты (xUnit):** Покрывание доменных сущностей и **UseCase** классов.
- **Стратегия логирования (Serilog):** Структурированное логирование в консоль и/или файл. Сообщения должны содержать контекст (например, **PhotoId**, **UseCaseName**). Ошибки логируются с сохранением оригинальных кодов из справочников **Errors**.
- **Обработка бизнес-ошибок (ROP):** Обработка ошибок производится с помощью паттерна Result. Использование реестров ошибок для каждого слоя:
 - **DomainErrors**: бизнес-правила (например, **DomainErrors.Tag.TooLong**).
 - **ApplicationErrors**: ошибки потока (например, **ApplicationErrors.Photo.FileNotFoundOnImport**).
 - **InfrastructureErrors**: системная грязь (например, **InfrastructureErrors.FileStorage.DiskFull**). Оборачивается в **Result** после записи в лог.
- **Защита инвариантов:** Доменная модель не позволяет перевести систему в невалидное состояние. Валидация происходит строго внутри **Domain**.

7. Этапы разработки

1. **Domain & Testing:** Создание сущностей (**Photo**, **Album**, **Folder**) с инкапсуляцией. Реализация возвратов паттерна **Result** / **ResultVoid** и написание Unit-тестов.
2. **Архитектурные тесты:** Настройка ArchUnitNET для фиксации слоев.
3. **Скелет БД:** Создание DDL-скриптов для генерации файла SQLite со всеми таблицами.
4. **Data Access & Infrastructure:** Конфигурация **Serilog** в DI-контейнере. Реализация чистой версии **UnitOfWork**, CRUD-операций и перехвата **Exception** с записью в лог.
5. **Use Cases:** Реализация прикладного слоя (**Application**), внедрение логгера (**ILogger<T>**) в Use Cases и оркестрация вызовов.

6. **Интеграционное тестирование:** Запуск In-Memory SQLite (или тестового файла БД), проверка полного цикла работы приложения и анализ сгенерированных файлов логов на предмет корректного отслеживания ошибок.